RISC-V Event Trace Specification

Authors: Bruce Ableidinger, MIPS

Version v0.0.0, 2026-05-22: Draft

Table of Contents

List of figures

List of tables

This document is in the Development state

Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.

Copyright and license information

Contributors

This RISC-V specification has been contributed to directly or indirectly by:

- Bruce Ableidinger <bableidinger@mips.com>
- Robert Chyla (robert.chyla@globalfoundries.com)

Chapter 1. Introduction

Event Trace is an extension to the RISC-V Instruction Trace that provides hardware support for selectively tracing key executed events, called Event Types. It enables the capture of detailed execution information, which can be used for debugging, performance analysis, and system monitoring.

Event Trace extends the existing Instruction Trace while also leveraging its common packet types and trace sink options.

The Event Trace extension introduces new memory-mapped registers for setting up read-only contigurations, dynamical selection of trace events to trace, submodes of operation, hardware filters, and accessing its running status.

Chapter 2. Event Trace Features

- Extension of ratified RISC-V Instruction Trace as an additional encoder mode with additional message types
- Based on same core hardware interface, now defined as RHTI (RISC-V Hart Trace Interface) [1]
- A trace mode that turns off full instruction trace and turns on hardware filtering of instruction types and external interrupt events
- Event types that are independently traceable allowing control of the granularity of information captured
- Event types that include function calls and returns, exceptions, interrupts, and context changes
- Event types that include a timestamp based on a common fixed-frequency trace clock to provide precise timing information for an execution timeline and for performance analysis
- Event types that can selectively include profiling performance counter (PPC) values to provide core performance information such as IPC (instructions per cycle) or cache misses, for the duration of the event
- Run-time filtering of events based on type, privilege level, operating system context, and user-instrumented directives in source code to reduce the volume of trace data and focus on relevant information
- External signals - edge-triggered signals that can insert a trace packet containing the closest retired PC at the time of the event, plus timestamp. This enables tracing of external events such as interrupts, I/O events, DMA start-end markers, uncore signals, or SoC-based events outside of program execution, and to time correlate them **with** program execution.
- Utilizes many of the capabilities and features of the existing RISC-V Instruction Trace thus ensuring compatibility to the existing Instruction Trace to reduce development costs and limit the hardware size of Event Trace extensions

2.1. Event Types

A summary of the events types that can be traced with Event Trace is shown in the table below. Each type can be independently enabled or disabled for tracing.

Table 1. Summary of Event Trace Event Types

Name	Description
Function Calls, Returns	Calls are categorized as Inferrable and Uninferrable types, specifically: itype = 8 (indirect call), 9 (direct call), 12 (co-routine swap), 13 (return)
Exceptions	Occurrence (itype=1) records PC of instruction causing exception, cause register, + exit address (itype=3 (trap return))
Interrupts	Occurrence (itype=2) records PC of instruction that was interrupted, cause register, + exit address (itype=3 (trap return))
Trap Return	Occurrence (itype=3) records PC of return-from-trap instruction (MRET or SRET) of an exception or interrupt. This itype distinguishes the event from a normal function return.
Context + Priv Level	Records **s/hcontext** register when it is written to, plus **privilege level** and indication of s or h context

Name	Description
Debug trigger/watchpoints	Records PC of instruction that hit the trigger, plus trigger ID and timestamp. Triggers and their comparison conditions are optional; they can include instruction retired at a specific address or range of addresses, data address read or write or access, data value, privilege mode, scontext, or other trigger conditions.
Periodic PC Sampling	Programmable timer that generates a trace packet with the most recent retired PC and timestamp when it expires. This allows for periodic PC sampling of the executing code to provide an execution profile over time.
External Signal (trigger) Events	Edge-triggered signals that can insert a trace packet containing the closest retired PC at the time of the event, plus timestamp. This enables tracing of external events such as interrupts, I/O events, DMA start-end markers, uncore signals, or user-selectable SoC-based events outside of core execution and to **time correlate** them with program execution. External Trace Triggers are defined in the Trace Control Interface spec [riscv-trace-control-interface-spec-1.0].
Performance Counter Selector Events	RISC-V HPM (Hardware Performance Monitor) [3] includes selector registers (mhpmeventX) which are user-programmed to select one (or several) core event types to count. This Event Trace feature extends this performance counter feature to record the closest retired PC on the rising edge of mhpmeventX output and save it in the trace buffer thus showing **where** in the code the core event occurred.

2.2. Event Trace Timestamp

The **Event Trace Timestamp** is the same timestamp timer used by Instruction Trace. It is a continuous running count-up counter (not a delta timer). As the RISC-V-Trace-Control-Interface specification describes in detail, there are optional sources for the timestamp: 1) External, 2) Internal System, 3) Internal Core, and 4) Shared.

If the (optional) Timestamp Unit is included in the Instruction Trace, then:

- **Timestamp enable**: must be enabled by setting **trTsControl.trTsActive**, **trTsControl.trTsEnable**, and started by setting **trTsControl.trTsCount**.
- **Timestamp off**: turned off, i.e. not included in trace packets (for Instruction AND Event Trace) by clearing **trTsControl.trTsEnable**. This can save some trace bandwidth and storage.
- **Timestamp reset**: timestamp register can be reset by setting, then clearing **trTsControl.trTsReset**. This is recommended before starting a trace session since lower-values timestamps can benefit trace bandwidth and storage, because of leading-zero suppression.

It is highly recommended that the timestamp clock run at or a close ratio to cpu clock in order to accurately time short functions and point-to-point events such as hardware triggers.

2.3. Event Trace PPCs - Profiling Performance Counters

Event Trace can include the option of Profiling Performance Counters (PPCs) which are delta counters that are recorded at the time of an event, simultaneously reset to zero, then begin counting again. This is a requirement to keep the counter values smaller to reduce trace bandwidth and storage. Post-processing of the trace data can reconstruct event counts across multiple event sequences to provide accumulated totals. An example would be inclusive function call-return counts, where snippets of code execute in a function then more counts for functions that it calls. "Self" or "exclusive" counts, i.e. only counts attributed to the code of the function itself would only accumulate counts from calls to other functions and from returns to calls of other functions.

2.4. Event Trace Features and Benefits (non-normative)

All Event Trace features are optional, however, the minimal set of event types recommended are: call/returns, interrupts, exceptions and context ID + privilege level.

Table 2. Event Trace Features and Benefits

Feature	Benefits
Function Calls, Returns	Trace history of function calls/returns assists in debugging and making performance measurements, all with zero execution overhead. Trace can be compressed and visualized as a flame graph. Timestamping provides complete timing analysis of function execution including min, max, and average durations and the number of times a function has been called. The addition of Profiling Performance Counters (PPCs) provides detailed microarchitectural information about the dynamics of processor efficiency and throughput such as IPC, cache misses, branch mispredicts, etc., with measurement granularity down to the function level.
Exceptions	Tracing occurrences of exceptions and their causes can be beneficial for locating illegal instructions, instruction or data misalignments (that slow down execution speed), and page faults. These types of code conditions may occur infrequently such that the may be difficult or impossible to trace with generic instruction trace. Exceptions occur whenever the processor undergoes privilege mode changes; tracing these can show when and for how long these privilege levels take to execute. Also includes is the tracing of Linux system calls.
Interrupts	Tracing occurrences of interrupts and their causes can be beneficial for observing correct interrupt priorities, nested interrupts, and the duration of interrupt handlers - especially critical in embedded systems.
PPCs - Profiling Performance Counters	PPCs provide detailed microarchitectural information about the dynamics of processor efficiency and throughput such as IPC, cache misses, branch mispredicts, stalls, etc. They can be used with Watchpoints to measure precise execution from point A to B of a code segments.
Context + Privilege Level	A high level view of execution can be captured by tracing context and privilege level changes. These show timing of each process and time executing the OS kernel as well as low-level hardware accesses in M-mode. A pie chart can show time in each privilege level. Waveforms can show the sequence of program and OS executions.
HW Filters for Privilege, Context ID, and Exception and Interrupt Cause	Filtering benefits the utilization of trace memory by recording only events that are relevant to the debugging or performance analysis task. For example, if the developer is only interested in user-level code, then filtering out events that occur in M-mode and S-mode can save trace memory and make post-processing more efficient. If the developer is only interested in a specific process, then filtering by context ID excludes all other processes, including the OS itself. If the user is only interested in specific exception types or interrupt causes, then the resulting display will show only relevant data.

Feature	Benefits
Debug trigger/watchpoints	Tracing hardware trigger/watchpoints [4 pp. chpt 5] are useful for programming markers without having to instrument and recompile code. They provide traces of specific locations in one's code for timing executions, counting occurrences, and especially for precisely measuring algorithic loop times, variations, and observing data-dependent behavior, such as cache misses or a function that executes in a varying amount of time based on the input data. For device driver debugging, watchpoints can be set on memory-mapped I/O addresses to trace reads or writes for debugging and performance throughput of I/O. These types of measurements are difficult or impossible to trace with generic instruction trace because of the infrequency of the events and the fact that they may be accessed by a number of different locations in the code.
Periodic PC Sampling	Periodic PC sampling provides statistical profiling of code execution over time, with no performance penalty that software-based sampling incurs. This mode can very quickly identify "hot spots" in the code as candidates for optimization. With the addition of profiling performance counts, this trace mode provides details of program conditions accumulated at each of the sample addresses.
External Signal (Trigger) Events	Applicable for embedded systems, tracing external events such as the rising edge of an interrupt signal, the closest concurrent retired instruction along with tracing the cpu processing of the interrupt - this combination provides a means of precisely measuring interrupt latencies. Capturing many of these will characterize the timing and variability of interrupt processing and could expose maximum values that violate real-time requirements. External signals can also be traced in a logic analyzer mode and shown as timing waveforms to provide visibility into signals and timing that may otherwise be impossible to observe.
Performance Counter Selector Events	Normally, performance counter selector registers (mhpmeventX) are used to select one or more core events to count. With this Event Trace feature, the closest retired PC is recorded in the trace buffer on the occurrence of the mhpmeventX output signal. This shows **where** in the code the core event occurred; post-processing these addresses into module/function/line numbers provides direct user feedback as to the code locations where performance bottlenecks are occurring. For cpu events that can occur in high-frequency bursts such as cache misses, the trace packet captures a count of the number of occurrences of the event over a user-selectable window of time.

Chapter 3. Event Trace Feature Details

3.1. Non-function Events

3.1.1. Event Trace Message Fields

F-ADDR *means full or uncompressed address.* **C-ADDR** *means compressed address which the trace decoder converts into a full address from a previous message, starting from a sync message that includes a full address. A C-ADDR, for N-Trace [5], is called a U-ADDR (U=unique) which is generated with "previous PC XOR current PC" to compress. For E-Trace [6], compressed addresses (or delta address mode) are referred to as a differential encoding which is hardware logic that does "previous address - current address".*

Table 3. Event Trace Message Fields for non-function events

Event	EVTPC	EVTDATA	ITYPE	Description
Sync Enable	PC	SYNC	X	SYNC field: EVTPC is always F-ADDR Causes for Sync Enable: * Exit from Reset * Exit from debug. PC=first instruction to execute after debug mode. * Event trace enable. PC=instruction responsible for trace-on.
Sync Disable	PC	EVCODE	X	EVCODE field: EVTPC is always F-ADDR Causes for Sync Disable: * Enter-debug. PC=address loaded into depc (has not executed). * Event trace disabled. PC=instruction responsible for trace-off. * Enter-reset. PC=last instruction to execute before reset.
Exception	PC	cause	1	PC (C-Addr) is what is loaded into xepc (the address of the instruction that caused the exception), **cause** is what is loaded into the Exception Code field of xcause. cause signals are defined in Table 1 of RHTI spec [1].
Interrupt	PC	cause	2	PC (C-Addr) is loaded into xepc, **cause** is what is loaded into the Exception Code field of xcause.
Return from trap	PC	PCdest	3	Return from exception or interrupt (MRET or SRET) (to distinguish from normal function return) PC and PCdest (both are C-ADDR) are both generated when they are not filtered by a privilege mode filter.
Return from trap	------	PCdest	3	(Variant of above) Return from exception or interrupt (MRET or SRET). Generated when the source PC is filtered by privilege mode (but PCdest which is a C-ADDR is not).

3.2. Function Events

The following table lists the message types to support function Calls and Returns, with different types and numbers of parameters.

Table 4. Event Trace Message Types for Calls and Returns

Name	Source	Destination	ITYPE	Description
CALL_SYNC	PC (F-ADDR)	dest (C-ADDR)	8 or 9	Traces both src and dest addresses. The second address can be compressed. Full (absolute) timestamp (if TS enabled).
CALL_FULL	PC (C-ADDR)	dest (C-ADDR)	8 or 9	Traces both src and dest compressed (unique) addrs; relative timestamp (if TS enabled).
CALL_DEST	------	dest (C-ADDR)	8	itype=8 means indirect (uninferrable) call or co-routine swap. Source function can be derived from trace history however src address (i.e. line # of caller) is unknown.
CALL_PC	PC (C-ADDR)	------	9	inferrable call so dest obtained from opcode if binary available
RET_FULL	PC (C-ADDR)	dest (C-ADDR)	13	Traces both src and dest (return to) addresses
RET_DEST	------	dest (C-ADDR)	13	return-to address; caller (function) determined from symbolic address range of destination address and/or from history.
RET	------	------	13	Function returned to is determined from history. If caller was CALL_PC (inferrable caller), decoder can determine return destination address from 2-byte or 4-byte opcode.

If Calls/Returns are supported, Event Trace requires implementation of CALL_SYNC, CALL_FULL, and RET_FULL. CALL_DEST and CALL_PC are useful for reducing bandwidth. RET_DEST and RET are also useful for reducing bandwidth.

3.3. Context + Privilege Level Events

Instruction Trace already has the capability to trace changes in context ID and privilege level. Event Trace will utilize the same message type to trace these events - for N-Trace [5] and E-Trace [6].

ORing of controls for enabling Context and Privilege level

- Set **trTeControl.trTeContext** of Instruction Trace to enable tracing of Context changes, or
- Set Event Trace Context enable control bit

3.4. Debug Trigger/Watchpoint Events

A Debug Module can (and most often does) include a Trigger Module [4 pp. chpt 5] - hardware which matches on core conditions such as instruction address, data address, data value, privilege mode, context ID, etc. The Trigger Module can be programmed to generate an action called **Trace-notify** [4 pp. Table 12] using the value 4 for the 4-bit action field which is then routed to the RHTI [1] defined **trigger [2]** signal. Refer to Section 7.1 section for details.

For Event Trace, the **Trace-notify** action, when programmed by the user and enabled in Event Trace, results in a packet message output that includes the PC of the instruction that hit the trigger/watchpoint and a timestamp, plus PPCs if enabled. Thus Event Trace will record these Debug Triggers, and depending on the specific implementation of hardware debug triggers, can be amarker for instruction

address/addresses executed or when certain data addresses have been accessed.

The RHTI Extensions section Section 7.1 defines the **triggerID** signals for identifying which programmed trigger/watchpoint condition was met. The **triggerID** value will be included in the trace message to identify which trigger/watchpoint caused the trace event.

If the triggerID is not provided, trace decoder software may be able to identify and fill in the triggerID in the trace output. This first requires saving the trigger comparator setup for each trigger - primarily the execution or data address comparison value. The trace decoder, with this information, can then match the recorded PC in the trace message to the list of programmed trigger values and fill in the triggerID.

3.5. Periodic PC Sampling Events

For the **Periodic PC sampling** event type, a programmable counter runs until it expires which generates a trace packet with the most recent retired PC and timestamp then is preloaded with the count and counts down again. This allows for periodic PC sampling of the executing code to provide an execution profile over time. Note that with the inclusion of profiling performance counters (PPCs), this trace mode provides details of program conditions accumulated at the sample addresses.

While Instruction Trace provides a counter for periodic sampling, its range does not the best match for Event Trace. Therefore Event Trace will provide its own counter for periodic PC sampling. It uses the same powers-of-2 (exponential) sample count, with a count of **2^(<value>+6)**. With a 4-bit <value> selection field (0 to 15), the range extends from 2^6 to 2^21, or 64 counts to 2 million counts. Assuming counts = cpu cycles, the range is 64 cycles (64ns at 1GHz) to 2 million cycles (2ms at 1GHz) which is practical for sampling code execution.

Randomizing PC sampling. To avoid aliasing of the periodic PC sampling of a repeating code execution pattern, hardware can randomize the sampling interval by including an additonal random count after the powers-of-2 count has counted down. One method is to use a linear feedback shift register (LFSR) to generate a pseudo-random number which is used to pre-load a final count-down counter.

The number of randomizing bits, i.e. width of the LFSR is developer-selected. (Six is practical). Note that for shorter intervals the randomizing has a large percentage effect on the interval than for longer interval durations.

3.6. External Signal Events

Features to support:

- 8 external input signals, also called triggers or channels
- by default, programmed to recognize the rising edge on an input signal and generate a packet
- (optional) user-programmed to recognize a falling edge
- (optional) user-programmed to recognize either a rising or falling edge on the input signal, useful for timing the duration of a signal pulse.
- (optional) include a **cycles-window** to compact multiple edges into one trace packet. This is to prevent overloading the trace system with too many events in a short period of time when the signal is changing frequently. The logic, after desired edge is detected, begins the packet creation by latching the

edge value and timestamp. If, during the cycles-window, one or more additional signal edges occur, the packet records the signal as having multiple edges. When the cycles-window expires, the packet is formatted and output to the trace sink.

The post-processing visualization (e.g. logic analyzer-like waveforsm) shows the multi-edge packet as a "fuzzy" unit meaning it represents > 1 edge. The cycles-window count would indicate the relative width of this "fuzzy" edge. The recorded timestamp, however, does show the time location of the starting edge of the signal.

- **cycles-window** uses an 8-bit countdown counter which is pre-programmed by the user for 1 to 255 cycle duration.
- implement a trace packet that records 1) rising edge or 2) falling edge, or 3) either edge
  - which value recorded is dependent on user request for which edge/edges) to trace
  - or 4) > 2 edges within the time window
- provide two trace packet types:
  - rising, falling, both, or multiple edges occurred + channel ID + timestamp at the time the initial edge was observed. (NOT recording PC when it is not needed saves bandwidth and storage).
  - rising, falling, both, or multiple edges occurred, channel ID, timestamp, and **PC** of the closest retired instruction at the time the edge was observed
- enable/disable the recording of each input signal, independently
- multiplexer selector control to support up to 1 of 64 sets of 8 inputs into the hardware trace logic

The implementer must be aware that each channel is independent and has its own edge detection, cycle window, latching of edge state, timestamp and PC (if enabled). Events like a rising edge could occur simultaneously on multiple channels, including all channels. The trace encoder must serialize these events into a packet stream and into trace storage.

3.7. Performance Counter Selector-Based Events

RISC-V programmable performance counters have selector registers (programmed with the mhpmeventX CSRs) which are user-programmed to select one (or several) event types to count. (The 'X' of mhpmeventX are numbered 3-31). The selector outputs a "1" level when a certain countable event occurs such as a cache miss, branch mispredict, or any other countable event.

Single event vs. multi-cycle duration. Some counter event types are singular events - they are true for one cpu cycle indicating an occurrence. Some event types are true for multiple cycles indicating a condition that stays true for a duration, where a performance counter accumulates that number of cycles while the condition is true.

Level-to-Edge Conversion. Assuming selector outputs are level sensitive, the Event Trace hardware requires edge sensitivity, i.e. it needs to convert the level to an edge and only record the occurrence of the event once, even when it remains high for multiple cycles. (Another way of describing this is to convert a multi-cycle level into a single-cycle pulse).

Stretching logic high levels. Event Trace hardware must accommodate signals that can change level on every CPU cycle. If the clock used for the Trace Encoder runs at a divided ratio to the CPU clock, then the Event Trace hardware may require stretch logic to convert single-cpu-cycle levels into a level recognizable by the edge detector.

Single value output. Another programmable performance counter attribute is that some events report greater than 1 value for a single cycle. This can occur, for example, when counting retired instructions or even a class of instructions like loads that may retire more than one per cycle. A simplification for Event Trace is to treat all selector outputs as a single value of 0 or 1 (converting > 1 to 1 which can be done with an OR gate). An asserted selector output can be thought of as a code marker.

Tracing perf counter events. These (hpmeventX) selector outputs are routed to the Event Trace hardware to, if enabled, generate a trace packet. The trace packet latches the most recent retired instruction PC and timestamp. This enables tracing of where in the executing code a performance event occurs such as a cache miss, branch mispredict, or any other countable event that can be selected by a performance counter selector.

Enabling selector recording. An Event Trace control register (refer to Control Registers section) includes an enable bit for each performance counter selector included in Event Trace.

(optional) Many prog. perf counter events can occur with high frequency. To avoid overloading the trace system, hardware can include a digital "low-pass filer" to block highly frequent edges. This utilizes a one-shot triggered timer that, while true, blocks additonal event generation. The one-shot triggered timer duration is the same for all selector outputs. The lock-out timer logic also includes a counter that accumulates edge counts for the duration. When the lock-out timer ends, Event Trace constructs a packet that include the selector ID, the latched timestamp when the first event occurred, and the count.

The width of the timer lock-out counter is user selectable. Recommended is a width of 18 to 24 bits (increments-of-6 work best for N-Trace [5]), and the counter should be saturating so that it does not roll over and cause loss of information. When the lock-out timer ends, if the count is at maximum, this indicates in the trace packet that the count exceeded the maximum.

Limiting Event Trace Bandwidth based on Trace Encoder Feedback. Most Trace Encoders include a FIFO for buffering trace events and to smooth out bursts of trace recordings that can occur at higher rates than the back-end rate of streaming trace packets to memory or PIB (probe interface block). Most FIFOs have a buffer depth signal indicating when it is close to full (for example, 3/4 full). This indicator can act as a back-pressure signal to the Event Trace hardware to temporarily stop the generation of trace packets. This allows the trace system to catch up and avoid losing trace information due to FIFO overflow. When the back-pressure signal is deasserted, Event Trace generation can resume. For events like watchpoints, programmable performance counter selectors, and external signals which can occur at higher frequency, this feedback mechanism can be built into its hardware to either compress the trace information or simply throw away or reject these events.

Event Trace will also include the ability to set a **precedence for event types** so that if the back-pressure signal is asserted, the Event Trace hardware determines which events to prioritize for recording and which events to reject. For example, the developer may choose to prioritize the recording of function call/return events and reject watchpoint events when the back-pressure signal is asserted.

Control registers for perf counter selectors. Event trace control register includes a Read-Only field that specifies how many performance counter selectors are included in the Event Trace implementation. Of course the maximum is 29 since that is the maximum number of potential programmable performance counter selectors, defined in the RISC-V privileged spec. A practical number is, however, between 2 and 4.

Chapter 4. Event Trace Profiling Performance Counters (PPCs)

- PPCs are defined in Event Trace as delta counters, counters that are recorded at the time of an event, simultaneously reset to zero, then begin counting again. This is a requirement to keep the counter values smaller to reduce trace bandwidth and storage. Post-processing of the trace data can reconstruct event counts across multiple event sequences to provide accumulated totals. An example would be inclusive function call-return counts, where snippets of code execute in a function then more counts for functions that it calls. "Self" or "exclusive" counts, i.e. only counts attributed to the code of the function itself would only accumulate counts from calls to other functions and from returns to calls of other functions.

Two mechanisms for implementing delta counters are:

- Delta counters based on hpmcounters, or
- Delta counters based on independent (Event Trace) counters, i.e. not reliant on hpmcounters

4.1. Alternative 1: Delta counters based on hpmcounters

Implementers have the option to implement either of these mechanisms. The first mechanism may appear to save hardware costs by using existing hpmcounters, however the disadvantages can be:

1) delta counts are generated when a new event occurs, which terminates the previous event and the delta count is assigned to that terminated event. Generating delta counts requires additional latching of the running performance counter at the end of the terminating event and the beginning of the next event so it can be subtracted from the counter at the time that event terminates, to produce the delta count.

2) these counters become a shared resource between traditional software-controlled performance counters and their use with Event Trace.

3) hpmcounters are CSRs and located in the core. To implement delta counts requires either routing the outputs of each of these counters to the Trace Encoder to latch and subtract the start and end values or, adding this logic into the core and routing the delta count to the Trace Encoder. Both options require additional wires and routing complexity.

No additional control registers are required for this configuration since the hpmcounters are being used to generate the delta counts.

4.2. Alternative 2: Delta counters based on independent (Event Trace) counters, not reliant on hpmcounters

For Event Trace, independent profiling performance counters can be implemented as delta counters, with the output latched then reset to zero at the start of the next event. These delta counters should saturate at their maximum (usually all-ones value) to avoid roll-over.

Alternative 2 independent PPCs can also implement their own event selectors or use the existing hpmcounter event selectors by routing the outputs of the hpmcounter event selectors to the Event Trace unit (see [Performance Counter Selector Output]). The latter option saves hardware costs but requires the sharing of performance counter resources. Note that this sharing still allows for performance counters to be used at the same time as Event Trace uses them. It just means that the settings for these hpmevent

selectors have to be the same for both uses.

The previous section covered the Event trace feature "Event Trace Performance Counter Selector-Based Events" which generates a trace packet when an enabled performance counter selector output is a "1" level. The PPCs are a way of counting selector events and capturing the summed counts at the termination of an event, thus compacting these occurrences into a count. If this feature is implemented then the same selector outputs that generate Event Trace Performance Counter Selector-Based Events can also be routed to the PPCs to count the occurrences of these events between Event Trace events.

4.2.1. Recommended features of Profiling Performance Counters (PPCs) for Event Trace:

- 24-bit delta counters
- the number of counters implemented is in pairs of 2, thus the number of PPCs is 2, 4, 6, or 8
- counters are saturating, i.e. stop counting at all-ones max value until the next event occurs and resets each counter to zero. This provides a max count of 2^24, i.e. 16 million
- map each PPC to a MMIO address to accommodate testing and DV of the hardware.
- as a secondary use, provide a control register field to latch PPCs thus allowing JTAG control and readout of PPCs periodically. Since they are memory-mapped, this performance counter sampling can be done with no cpu intervention which is not possible with CSR-based performance counters.

4.3. Fixed Counter - Instructions Retired

A very common and useful performance measurement is IPC - instructions per cycle. This ratio indicates the instruction throughput of the core and identifies the actual code performance relative to the ideal. For example, if a core is a 2-issue/2-instruction retire per cycle core then its ideal and maximum performance would be IPC=2. Numbers less than 2 indicate how far from the ideal it is performing. Measuring IPC per function with Event Trace provides a reasonably-fine granular view of performance, and sorting function results by IPC will identify which functions are furthest from the ideal and thus have the most to gain by improving their performance.

The current RHTI specification [1] does not include a signal set for reporting the number of instructions retired in each cycle. The proposal for adding this signal set is here - Instruction Retired Count.

An implementer can include the **Fixed counter - Instructions Retired** feature without the RHTI spec by accessing and routing these signals from the HPM unit to the Trace Encoder.

Counting instructions retired is a RISC-V requirement, which is the **minstret** *CSR (instructions retired is hpmcounter1), a 64b performance counter (section 3.1.10 Hardware Performance Monitor, RISC-V Privileged Architecture Manual). In other words, it is already implemented in the core. The signal set simply needs to be routed to the Event Trace Encoder.*

If the developer is using Alternative 2 (independent counters), one choice is to dedicate the first 24-bit counter for counting retired instructions between events, with the same delta saturating counter mechanism as other PPCs. This provides a max count of 16 million instructions between events. This counter must be able to count > 1 instruction per cycle if the core retires more than 1 instruction per cycle. One disadvantage of this is that the **minstret** counter does not support count filtering by privilege level whereas a programmable counter can, if a selector (mhpmeventX) has a value that includes counting instructions retired.

Counting cycles. *For IPC ratio calculations, the number of cycles between events is also required. The timestamp may or* **may not** *be counting cpu cycles. If it does not, the Event Trace Decoder must be informed what the fixed ratio of timestamp ticks to CPU cycles is so it can convert timestamp counts into cycle counts. One hardware method for recording the timestamp-to-cycle ratio is to implement the Time Sync Message.*

Chapter 5. Event Trace-specific Hardware Filtering

5.1. Leaf Function Filtering

Leaf function filtering is the hardware filtering of call-return pairs which meet a time duration "less-than N cycles" threshold in order to reduce trace bandwidth and save storage for short functions that are less interesting to trace.

More documentation to follow.

Chapter 6. Event Trace Control Registers

6.1. Control Registers, Summary

Event Trace as a trace mode (separate from Instruction Trace) is set by software writing trTeControl.trTeInstMode=4 which is defined in the N-Trace spec [5 pp. Protocol defined trace mode].

Event Trace register starting point is an offset from the Trace Encoder base address. The Event Trace allocates and uses addresses **0x100-0x1FF**. Thus trTeEvtBase = trBaseEncoder+0x100. In the table below, the Offset is from the trTeEvtBase.

Table 5. Event Trace Control Registers (Summary)

Name	Offset	Description
trTeEvtControl	0x0	* EVT tracing status * EVT trigger enable * call/return modes * Event per-type enables
trTeEvtFeatures	0x04	Defines which Event Trace features are implemented, and sizes.
trTeEvtPPCEnables	0x08	Defines which Event Types output Profiling Performance Counter (PPC) values. Sets the number of PPCs output with each Event Type enabled.
trTeEvtPPCControl	0x0C	Control register that, when written to, causes all PPCs to be latched. Software can then read out all the PPC values. Only active when Event Trace is disabled.
trTeEvtPPCEvent*i*	0x10-0x48	PPC Event Selector Registers (if developer chooses independent selectors vs using hpmeventX selectors)
trTeEvtPPCCounter*i*	0x50-0x98	PPC Counter access registers (if developer chooses Alternative 2 - independent profiling perf counters)
trTeEvtExtSigControl	0xA0	Control register that enables up to 8 external signals for tracing signal transitions (edges)
trTeEvtPerfSelectControl	0xA4	Control register to enable tracing of performance counter selector events and setting time window for counting occurrences inside window.

6.2. Event Trace Control Registers (Details)

6.2.1. trTeEvtControl: Event Trace Control Register (trBaseEncoder+0x100)

Table 6. Event Trace Control Register fields for **trTeEvtControl** *(32b)*

Bits	Field	Description	RW	Reset
0	trTeEvtImplemented	Read as 1 if Event Trace is implemented	RO	1

Bits	Field	Description	RW	Reset
1	trTeEvtTracing	(Read) EVT **status**: 0: not tracing; 1:event trace is running. HW Triggers (in the Trigger Module)can enable or disable EVT. (Write) EVT **control**: When **trTeEnableTracing** = 1, writing 0 will disable trace that then can be enabled with a HW Trigger. For example, software can set up EVT disabled then program a HW trigger to match an executed address at the beginning of a program to start it. Writing 1 will turn EVT on. Note that a change in value will generate a trace-on or trace-off EVT message packet.	RW	0
2	trTeEvtTrigEn	1: Allows **trTeEvtTracing** to be set or cleared by Trace-on and Trace-off signals generated by the corresponding **Trigger Module**.	RW	0
4:3	trTeEvtCallMode	0: use messages that include both source and destination addresses for calls(CALL_SYNC, CALL_FULL). These generate more bytes (than modes 1, 2), however the decoder can determine the symbolic line # where calls are made and returned to without requiring a binary image. (Required if Call/Ret implemented) 1: Generate CALL_DEST for both direct and indirect calls. 2: least bandwidth: use CALL_DEST for indirect calls, CALL_PC for direct (inferrable) calls. Most aggressive on saving bandwidth 3: reserved	WARL	0
6:5	trTeEvtRetMode	0: use messages that include both source and destination addresses for returns (RET_FULL). These generate more bytes (than modes 1, 2), however the decoder can determine the symbolic line # where return is made and returned to without requiring a binary image. (Required if Call/Ret implemented) 1: generate RET_DEST for returns. 2: least bandwidth: use RET with no addresses when return address matches the call, otherwise RET_DEST. 3: reserved	WARL	0
7	RESERVED			
8	trTeEvtExceptionEn	0: don't trace; 1: trace exceptions at the source.	RW	0
9	trTeEvtInterruptEn	0: don't trace; 1: trace interrupts at the source.	RW	0
10	trTeEvtCallRetEn	0: don't trace; 1: trace calls and returns	RW	0
11	trTeEvtSContextPrivEn	0: don't trace; 1: trace when Supervisor Context (CSR 0x5a8) is written to. Note: Context writes are also enabled when the **trTeControl.trTeContext** bit is set (only mentioned here).	RW	0
12	trTeEvtHContextPrivEn	0: don't trace; 1: trace when Hypervisor Context (CSR 0x6a8) is written to.	RW	0
13	trTeEvtTrapReturnEn	0: don't trace; 1: trace interrupts and trap returns (hardware does not distinguish exception and interrupt returns)	RW	0
14	trTeEvtDbgTrigEn	0: don't trace; 1: trace debug trigger/watchpoints. The user sets up debugger hardware triggers with the Action value of 4 (Trace notify) that generates a trace message when the trigger/watchpoint matches. This control bit enables or disables the generation of these trace messages.	RW	0

Bits	Field	Description	RW	Reset
15	trTeEvtPCSampEn	0: don't trace; 1: enable the tracing of periodic PC samples. The user sets up period of sampling and optionally the inclusion of randomizing the period.	RW	0
16	trTeEvtPCSampRandEn	0: do not enable randomizing of sample period; 1: enable randomizing of sample period. The random count is added to the powers-of-2 count down set by the user. An LFSR is used to generate the random count. The width of the random count is user selectable (recommended at 6 bits), but fixed at design time.	RW	0
17	trTeEvtExtTrigEn	0: don't trace; 1: enable the tracing of external trigger signals. The user sets up which external signals to trace and the edge detection (rising, falling, or both) for each signal.	RW	0
18	trTeEvtPerfCtrSelEn	0: don't trace; 1: enable the tracing of performance counter selector outputs. The user first programs the hpmeventX registers that generate the outputs to trace.	RW	0
23:19	Reserved	Reserved for future Event Type enables		
27:24	trTeEvtSampDuration	Periodic sample duration for PC sampling events. Interval duration is exponential in powers of 2 - cycles = 2^(trTeEvtSamp Duration+6). i.e. range is 2^6 to 2^21 (64 to ~2 million cycles). For a 1GHz clock, the range is 64ns to 2ms.	WARL	0
31:28	Reserved			

For the setup of **trTeEvtExceptionEn** *and* **trTeEvtInterruptEn**, *the return from an exception or interrupt are common and called a "trap return". Hardware tracing cannot distinguish between a return caused by an interrupt or an exception so if one or the other is turned off with these control bits, all trap returns will still be generated as EVT packets. The Trace decoder must deal with trap returns that may not have an associated exception or interrupt event. Privilege level filtering can be used to selectively quash some of them. The user may also choose to turn off the tracing of trap returns separately.*

6.2.2. trTeEvtFeatures: Event Trace Features RO Register (trBaseEncoder+0x104)

Table 7. Event Trace Features Read-only Register - **trTeEvtFeatures** *(32b)*

Bits	Field	Description	RW	Reset
0	trTeEvtIntExcImp	Read as 1 if Interrupt and Exception event types are implemented	RO	1 (SD)
1	trTeEvtCallRetImp	Read as 1 if Call and Return eventtypes are implemented	RO	1 (SD)
2	trTeEvtContextPrivImp	Read as 1 if Context and Privilege event types are implemented	RO	1 (SD)
3	trTeEvtDbgTrigImp	Read as 1 if Debug Trigger event types are implemented	RO	1 (SD)
4	trTeEvtPCSampImp	Read as 1 if PC Sampling is implemented	RO	1 (SD)
5	trTeEvtExtTrigImp	Read as 1 if External Signal (Trigger) event types are implemented	RO	1 (SD)
6	trTeEvtPerfCtrSelImp	Read as 1 if Performance Counter Selector event types are implemented	RO	1 (SD)
10:7	trTeEvtPPCsImp	Number of Profiling Performance Counters implemented (0-15)	RO	8 (SD)
11	trTeEvtPPCtype	Type of Profiling Performance Counters implemented: 0: independent counters; 1: based on hpmcounters	RO	0 (SD)

Bits	Field	Description	RW	Reset
12	trTeEvtPPCSelType	Type of Profiling Performance Counter selectors implemented: 0: independent selectors; 1: based on hpmeventX	RO	0 (SD)
31:13	RESERVED			

SD = System Dependent. Reset value of a field is dependent on the build.

6.2.3. trTeEvtPPCEnables: Event Trace Profiling Performance Counter (PPC) Enable Register (trBaseEncoder+0x108)

Each Event Type can be separately programmed to output PPC values when that Event Type is enabled. This register defines the enable bit for each Event Type to output PPC values in a packet immediately after the event.

The value of the field is always 0 if the feature is not implemented, thus by writing 1 to each field, the WARL return value indicates if the feature is implemented. The reset value is also 0 for all fields which means that by default no Event Types will output PPC values until the user enables them.

Table 8. Event Trace PPC Enables Register - **trTeEvtPPCEnables** *(32b)*

Bits	Field	Description	RW	Reset
0	trTeEvtExcPPC	0: don't trace PPCs; 1: trace PPCs for Exceptions	WARL	0
1	trTeEvtIntPPC	0: don't trace PPCs; 1: trace PPCs for Interrupts	WARL	0
2	trTeEvtCallRetPPC	0: don't trace PPCs; 1: trace PPCs for Calls and Returns	WARL	0
3	trTeEvtSContextPrivPPC	0: don't trace PPCs; 1: trace PPCs for Supervisor Context and Privilege mode changes	WARL	0
4	trTeEvtHContextPrivPPC	0: don't trace PPCs; 1: trace PPCs for Hypervisor Context and Privilege mode changes	WARL	0
5	trTeEvtTrapRetPPC	0: don't trace PPCs; 1: trace PPCs for Trap Returns	WARL	0
6	trTeEvtDbgTrigPPC	0: don't trace PPCs; 1: trace PPCs for Debug Trigger (Watchpoint) occurrences	WARL	0
7	trTeEvtPCSampPPC	0: don't trace PPCs; 1: trace PPCs for PC sampling events	WARL	0
8	trTeEvtExtSigPPC	0: don't trace PPCs; 1: trace PPCs for External Signal (Trigger) events	WARL	0
9	trTeEvtPerfCtrSelPPC	0: don't trace PPCs; 1: trace PPCs for Performance Counter Selector events	WARL	0
15:10	RESERVED			
19:16	trTeEvtPPCNumTrace	0: turn off tracing of PPCs for all Event Types 1-15: Trace this number of PPCs. This tells the Encoder how many counters to include in the PPC trace packet. If the value is greater than the number of PPCs implemented, then trace all PPCs.	WARL	0
31:20	RESERVED			

6.2.4. trTeEvtPPCControl: Event Trace Profiling Performance Counter (PPC) Control Register (trBaseEncoder+0x10C)

A write to the **trTeEvtPPCControl** register (any value) causes all the PPCs, i.e. **trTeEvtPPCCounter*i***, to be snapshotted (latched) allowing their respective values to be read by software. The PPCs are reset to zero and begin counting the pre-programmed events on the next cycle.

This action only works if Event Trace is disabled.

6.3. Event Trace PPC Event Selector Registers - trTeEvtPPCEvent*i* (64b) (trBaseEncoder+0x110-0x148)

The **trTeEvtPPCEventi** registers are equivalent to the **hpmeventX** selectors of the HPM (Hardware Performance Monitor). This set of registers is based on the PPC Alternative 2 (previously described in this document) where the PPCs are independent Profiling Performance counters.

The EVT developer has the choice of using hpmeventX selectors (components of the Hardware Performance Monitor (HPM)) or implementing PPCEvent*i* selectors which are independent of the HPM (Hardware Performance Monitor). Each PPCEvent*i* selector register is 64 bits and is identical programmatic-wise to hpmeventX selectors of the HPM. (An exception is that the interrupt bit 63 status/control bit is undefined). These registers can be read/written as 64-bit registers or as address-adjacent 32-bit low/high registers.

The Event Trace specification and associated memory mapping is assuming the maximum number of PPCs is 8. Thus the *i* in the register name is an index from 0 to 7. If the implementation includes fewer than 8 PPCs, then the unused register slots are reserved.

Table 9. Event Trace PPC Event Selector Register **trTeEvtPPCEventi** *fields*

Bits	Field	Description	RW	Reset
[31:0]	Low	Lower 32 bits of PPC event selector. (address 0x110, 0x118, 0x120, 0x128, 0x130, 0x138, 0x140, or 0x148)	RW	0
[63:32]	High	Upper 32 bits of PPC event selector. (address 0x114, 0x11C, 0x124, 0x12C, 0x134, 0x13C, 0x144, or 0x14C)	RW	0

6.4. Event Trace PPC Counter Registers - trTeEvtPPCCounter*i* (24b) (trBaseEncoder+0x150-0x198)

This set of registers is only applicable if the implementation includes independent PPC counters (Alternative 2). The recommended width of each counter is 24 bits. They are delta, saturating counters that, for Event Trace, count performance events for the duration of each Event Trace event that is then packetized and sent into the trace stream. (Unfortunately the term "event" is an overloaded term in this context.)

This register access is an optional feature. If implemented, these registers are memory-mapped to allow for read/write access and that does not require cpu instructions to access them (as CSR-based HPM counters require).

The mapping of these registers to MMIO space is optional. The purposes are:

1. read/write access for DV-based testing, in-circuit diagnostic testing, and
2. to be used as independent (outside of Event Trace) perf counters. This extends the usefulness of these counters beyond Event Trace and HPM counters. For example, they can enable memory-based access

by another core in the cluster or by JTAG or other external debug port/device for taking performance measurements without cpu intervention.

The registers exist in the design as Capture registers. When a new Event Trace event occurs, each of the profiling performance counters must be latched in order for the counter to be zeroed and ready to begin counting on the next cycle. Then the Trace Encoder must create a message packet with each of the PPCs put into the packet, sequentially. This requires that all the PPCs be latched simultaneously.

If MMIO access is implemented, software can first snapshot the counters then read the Capture registers sequentially, until the next snapshot. The snapshot action is done with a write to a specific control register (s)

The **trTeEvtPPCCounter*i*** registers are similar to hpmcounterX counters of the HPM (Hardware Performance Monitor). This set of registers is based on the PPC Alternative 2 (previously described in this document) where the PPCs are independent Profiling Performance counters.

Table 10. Event Trace **trTeEvtPPCCounter*i*** *Capture Registers (24b)*

Addr Offset	Name	Size	RW	Reset	Description
0x150	PPCCounter0	[23:0]	RW	Undef	Counter 0 and 1 can be access as one 64b read with PPCCounter0 in the lower 32 bits and PPCCounter1 in the upper 32 bits.
0x154	PPCCounter1	[23:0]	RW	Undef	For 64b cores, this allows for more efficient reading of two counters with one read cycle.
0x158	PPCCounter2	[23:0]	RW	Undef	
0x15C	PPCCounter3	[23:0]	RW	Undef	
0x160	PPCCounter4	[23:0]	RW	Undef	
0x164	PPCCounter5	[23:0]	RW	Undef	
0x168	PPCCounter6	[23:0]	RW	Undef	
0x16C	PPCCounter7	[23:0]	RW	Undef	

6.4.1. trTeEvtExtSigControl: Event Trace External Signal Control Register (trBaseEncoder+0x1A0)

Event Trace supports up to 8 external signals that can be traced as events. The user programs the edge detection for each signal (don't trace, rising, falling, or both) in the **trTeEvtExtSigEnables** register. When the corresponding signal edge is detected, an Event Trace message is generated with the signal ID and the PPC values, if enabled.

These fields are WARL; the value of the field is always 0 if the feature is not implemented, thus by writing 1, 2, or 3 to each field, the WARL return value indicates if the feature is implemented. The reset value is also 0 for all fields which means that by default no external signals will be traced until the user enables them.

Table 11. Event Trace External Signal Control - **trTeEvtExtSigControl** *(32b)*

Bits	Field	Description	RW	Reset
1:0	trTeEvtExtSig0Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
3:2	trTeEvtExtSig1Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0

Bits	Field	Description	RW	Reset
5:4	trTeEvtExtSig2Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
7:6	trTeEvtExtSig3Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
9:8	trTeEvtExtSig4Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
11:10	trTeEvtExtSig5Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
13:12	trTeEvtExtSig6Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
15:14	trTeEvtExtSig7Edge	0: don't trace; 1: trace rising edge 2: trace falling edge 3: trace both edges	WARL	0
21:16	trTeEvtExtSigMux	Control field to select a developer-provided multiplexer of N groups of 8 signals (where N can be 1 to 64) external signal groups to trace. For example, if the developer has 32 external signals they want to be able to trace, the developer implements a mux with 4 input sets of 8 signals per set.	WARL	0
22	trTeEvtExtSigPCMode	0: Do not include the PC in the external signal trace message (to save on bandwidth) 1: Include PC in external signal trace message, to correlate transition of signal to instruction executed in core	RW	0
23	trTeEvtExtSigFIFOLockout	0: ignore Trace Encoder FIFO almost-full condition 1: when Trace Encoder FIFO feedback signal indicates "almost full", stop accepting external signal events. This is to prevent the Trace Encoder FIFO from filling up with external signal events and blocking the tracing of other more important events such as exceptions and interrupts (if enabled).	WARL	0
31:24	trTeEvtExtSigWindow	0-255: The **cycles-window** is like a "one-shot" circuit, triggered by the signal edge (user-selectable). While the cycle-window is open, any additional edges causes the recording state to change from "edge" to "multi-edge". 0: no cycles-window; 1-255: number of cycles for marking additional edges. This field is common for all signals.	RW	0

6.4.2. trTeEvtPerfSelectControl: Event Trace Performance Selector Control Register (trBaseEncoder+0x1A4)

The Event Trace Performance Selector feature supports the conversion of a HPM (Hardware Performance Monitor) selector (hpmeventX) output assertion into a Event message, and records the number of times this happens in a user-selectable time window. In other words, the occurrences of a standard performance counter input are also routed to the Trace Encoder to be recognized and counted as a trace event.

Tracing performance selector outputs is in some ways similar to the external signal trace, with the limitation of only tracing on rising edge of the selector output. It is also defines a time window to accumulate multiple occurrences of the perf event (which also reduces the frequency of EVT messages to a manageable amount since there are a number of perf selector events that can occur very frequently, for example counting load instructions).

The logic for this EVT event type requires a counter to count the number of rising edges occurring in the

lockout window. This becomes a performance measurement tool that records incremental counts of an event in an pseudo-periodical timeline where an initial event occurrence starts the count and the lockout time ends the count where it is recorded. The counter is saturating (stops counting when reaching maximum value). The recommended width is 24 bits and leading-zero suppression for recording the value in the trace packet.

The trace packet requires that the PC (address of latest retired instruction) and timestamp be latched at first occurrence of the selector event, the counter counts the high level during the time window, then the packet combines the PC, event count, and starting timestamp when the time window ends.

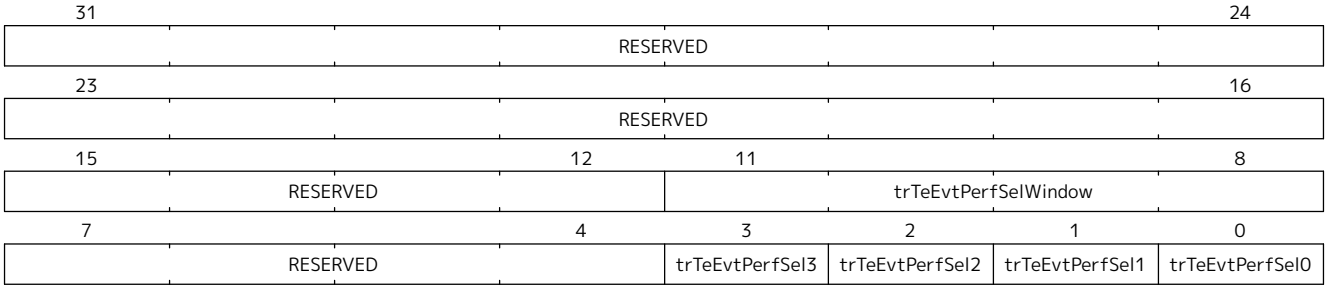

Figure 1. Event Trace Performance Selector Control - **trTeEvtPerfSelectControl** *(32b)*

Table 12. Event Trace Performance Selector Control - **trTeEvtPerfSelectControl** *(32b)*

Bits	Field	Description	RW	Reset
0	trTeEvtPerfSel0	0: don't trace; 1: trace rising edge of selector output; count level (high) during time window	WARL	0
1	trTeEvtPerfSel1	0: don't trace; 1: trace rising edge of selector output; count level (high) during time window	WARL	0
2	trTeEvtPerfSel2	0: don't trace; 1: trace rising edge of selector output; count level (high) during time window	WARL	0
3	trTeEvtPerfSel3	0: don't trace; 1: trace rising edge of selector output; count level (high) during time window	WARL	0
7:4	RESERVED			
11:8	trTeEvtPerfSelWindow	Cycles interval for counting perf selector level (high) events. Interval duration is exponential in powers of 2 **cycles = 2^(trTeEvtPerfSelWindow + 6)** Range is 2^6 to 2^21, i.e. 64 cycles to ~2 million cycles in powers-of-2 increments. For a 1GHz clock, the range is 64ns to 2ms. Common for all Perf Selects.	RW	0
31:12	RESERVED			

Chapter 7. RHTI Extensions for Event Trace

This file (RHTI-Extensions.adoc) defines extensions to the "RISC-V Hart to Trace Interface Specification" [1] to support several features for Event Trace. While an implementer can wire up these signals "on their own", RHTI defines them, along with all the existing signal definitions for Instruction Trace, to provide the named signals for Event Trace use.

7.1. Trigger/Watchpoint ID

As explained in the RHTI [1] section of the specification, the debug module of the RISC-V hart may have a "Trigger Unit", also named "Trigger Module" in the RISC-V Debug specification [4]. A 4-bit action field defines what happens when a trigger/watchpoint match occurs. Encoded codes 2-5 are for trace use. The value 4 is **Trace-notify**.

The RHTI defines trigger [2] signal to be **Trace-notify**. When asserted, it indicates a match on one of the programmed hw triggers.

The following signal definition extends the section "Optional sideband signals", Table 8 of the spec with the definition of **triggerID** which identifies **which** trigger matched.

Table 13. Optional sideband encoder **triggerID** *output signals*

Signal	Group	Function
triggerID	O	4-bit field that encodes the ID of the matching trigger thus supports up to 16 triggers. If multiple triggers match on the same cycle, hardware precedence must ensure that the lowest ID value is asserted on the encoded **triggerID** field (over higher ID encoded values). Simultaneous matches may occur with a core which retires more than one instruction per cycle or hw trigger comparisons overlap. (Also, there is nothing to prevent a user from programming multiple triggers with the same match condition.)

7.2. HPMeventX Output

*Table 14. Optional sideband event selector outputs *HPMEventX*for Event Trace Encoder*

Signal	Group	Function
HPMeventXOutput[*eventNum_p-1*:0]	O	Field that carries (routes) the least significant Num Hardware Performance Monitor (HPM) hpmEventX select outputs, where Num is *eventNum_p*. These signal outputs may be used for profiling performance counter delta counting or Selector-based event generation or both. If HPM mhpmeventX selectors are being used (Alternative 2), *eventNum_p* can range from 1 to 8.

7.3. Instruction Retired Count

Table 15. Optional sideband encoder **InstRetCount** *output signals*

Signal	Group	Function

instretcount[*countwidth_p-1*:0]	0	Number of instructions retired in this cycle. This is usable by trace to accumulate the total number of instructions retired at the time of a trace message. The width of this field is *countwidth_p* which can be as small as 1 and as large as the encoded max number of retired instructions in any one cycle. For example, if a core can retire up to 4 instructions per cycle, then a 3-bit width is required to encode the count of retired instructions from 0 to 4.

Chapter 8. Trace Control Extensions for Event Trace

This file (Trace-Control-Extensions.adoc) defines the extensions required to the "RISC-V-Trace-Control-Interface.pdf" spec to support Event Trace.

The Trace Control Interface spec, Table 4, has several reserved address ranges labeled "Reserved for future standard extension". Event Trace control registers starting point is an offset from the Trace Encoder base address. The Event Trace allocates and uses addresses **0x100-0x17F**. Thus **trTeEventBase** = **trBaseEncoder+0x100**.

8.1. Event Trace Control Register Mode Field

This document defines Event Trace as a sub-component of the Trace Encoder. Event Trace as a trace mode (separate from Instruction Trace) is set by software writing

trTeControl.trTeInstMode = 4

which is defined in the N-Trace spec as "**Protocol defined trace mode**".

8.2. Time Synchronization Message for simultaneously recording mtime & timestamp

This message is common for N-Trace and E-Trace Instruction Trace and its purpose is to record mtime and trace timestamp simultaneously for Trace message synchronization. This message is generated periodically based on a configurable interval of N- or E-Trace messages, and also aperiodically after certain events such as trace-on, exit debug, exit reset, ProgTraceSync message, and immediately after trace has stopped or halted.

MTIME_SYNC Periodic Generation The period for generating MTIME_SYNC is defined by a field in the register **trTsControl** inside the TimeStamp (TS) sub-component of Instruction Trace. The field, [14:10], of the ratified spec is Reserved. The lower 4 bits are, in this spec, being relabeled **trTsControl.trTsSyncMax**. This controls the interval for generating the MTIME_SYNC message based on a count of the number of N-Trace messages output.

Interval Count The interval count is 2^(7+trTsSyncMax) where **trTsSyncMax** usable values ranges from 1 to 15, where 0 is "**off**". This provides a range from 2^8 (256) to 2^22 (~4x10^6). Note that this field is WARL so some of the upper values could be left out to reduce hardware.

8.2.1. Register trTsControl: Timestamp Control Register Field

*Table 16. Timestamp Control Register Field - **trTsControl.TrTsSyncMax** (32b)*

Bits	Field	Description	RW	Reset
13:10	trTsSyncMax	**0**: MTIME_SYNC disabled **1-15**: Period for generating MTIME_SYNC message is 2^(7+trTsSyncMax) N-Trace messages	WARL	0

Engineering (non-normative) NOTE: MTIME_SYNC can be implemented by using a 16-input multiplexer; the first input selects always-0, the remaining 15 inputs connect to the appropriate divide-by-2 taps of the binary counter of N-Trace messages, starting at bit 7 and ending at bit 22, with the 4 bits of trTsSyncMax as the mux selector. This logic then generates an output pulse of MTIME_SYNC at the rising edge of the mux output. As mentioned, a narrower mux could be used in the actual implementation to save

on hardware. A 10-input mux would have a max period of 2^17, or 128K of N-Trace messages.

8.3. MTIME_SYNC Aperiodic Generation

If enabled, an MTIME_SYNC message shall be generated sometime after (when is not critical):

1. a trace-on, exit debug, or exit reset message
2. immediately after trace has stopped or halted (this needs to be timely)

This provides the time synchronization after a gap in trace and at the end of the trace buffer. The interval timer must also be reset.

Chapter 9. N-Trace Packets

This file (N-Trace-packets.adoc) defines the N-Trace [5] packets for Event Trace.

9.1. Time Synchronization Packet for recording mtime & timestamp

N-Trace MTIME_SYNC Message to record mtime and trace timestamp simultaneously for Timing Synchronization.

Table 17. N-Trace MTIME_SYNC Message for Timing Synchronization

Bits	Field	Description
6	TCODE	Value = 0x0F (15)
Cfg	SRC	Source Hart of this message (width is trTeSrcBits)
Var	mtime	F (Full) mtime value (64b w leading zero suppression)
Var	TSTAMP	F (Full) timestamp value (64b w leading zero suppression)

Chapter 10. E-Trace Packets

This file (E-Trace-packets.adoc) defines the E-Trace [6] packets for Event Trace.

Appendix A: Event Trace Operations (non-normative)

- Event Trace is a mode of the Trace Encoder which is set up with **trTeControl.trTeInstMode=4**. Then, **trTeControl.trTeEnable** is used to enable trace or disable and flush it, with **trTeControl.trTeEmpty** used to indicate when the flush has completed.
- **trTeEvtControl.trTeEvtTracing** is both a control and status bit. Reading it indicates Event Trace is tracing or not. Writing to it enables native software to start or stop Event Trace dynamically such as turning it off at the beginning of a function and turning it back on at the end, or visa versa.
- Event Trace can be started by native software writing to the Event Trace control registers. This can also be done with a JTAG command writing to the same registers.
- Event Trace can be first enabled via (for example) a hardware trigger set up to match on a program instruction address then, when the program-to-be-traced starts executing and the trigger match occurs, it in turn sets **trTeEvtControl.trTeEvtTracing**, and event trace starts.
- Once running, a hardware trigger previously set up to disable trace can, at the occurrence of a trigger match, turn Event Trace off which clears **trTeEvtControl.trTeEvtTracing**. Similar actions are common with Instruction Trace.
- An instrumented program can also clear or set **trTeEvtControl.trTeEvtTracing** to disable or enable Event Trace.
- Event Trace can be disabled by writing **trTeEvtControl.trTeEnable** = 0. This does a flush to the trace system.
- Trace can be set up to stop tracing on buffer full: **trRamControl.trRamStopOnWrap** = 1. When this occurs, Event Trace continues running, however no more trace messages are written to the trace buffer.
- A self-hosted trace analysis program such as Linux *perf* could control the starting and stopping of Event Trace.
- Trace hardware supports on-the-fly switching between Event Trace and Instruction Trace. Memory-mapped (MMIO) hardware control registers allow a user program to change modes of trace so a trace recording can have both instruction trace messages intermixed with event trace messages. This feature allows the tracing of a portion of one program/process full execution while also tracing kernel-level events such as going in and out of kernel mode and running other processes.

Appendix B: Event Trace Global and per-Event Filtering (non-normative)

These global and per-event filters are documented in the **RISC-V Trace Control Interface Specification**, section 6.3. These are documented here since they are already documented and the specification ratified.

There are several types of useful Event Trace filters that can be provided by hardware to reduce the volume of trace data and focus on relevant information. While these registers and associated hardware are optional, they are required to be included in the implementation of Trace hardware.

Global filters are:

- Privilege level filtering - only trace events that occur in a specific privilege level or set of privilege levels. This is useful for tracing events in user mode vs. kernel mode.
- Context ID filtering - only trace events that occur in a specific context ID or set of context IDs. This is useful for tracing events in a specific process or thread.

Per-event filters are:

- Cause register filtering - only trace events that occur with specific cause register values are traced. These are applicable to exception or interrupt Event Types.

Below, the 'i' in trTeFilter**i**Control (and others) is reference to the index of the register (0, 1, etc), which means the developer can instantiate multiple sets of filter registers.

B.1. Privilege Level Filtering Requirements

Table 18. Privilege Level Filtering Registers

Register	Register Fields	Description
trTeFilteriControl	**.trTeFilterEnable** **.trTeFilterMatchPrivilege**	Set = 1 Set = 1 All other fields set to 0
trTeFilteriMatchInst	**.trTeFilterMatchChoicePrivilege**	[7:0] bits to match Privilege levels list in table below

Table 19. Privilege level encoding (from RHTI spec)

Value	Description
0	U
1	S/HS
2	reserved
3	M
4	D (debug mode)
5	VU
6	VS
7	reserved

B.2. Context ID Filtering Requirements

To enable Event Trace on matching a single Context ID write value and disable Event Trace on a non-

match write to Context ID requires the Table setup below.

For **trTeCompjControl** and **trTeCompjMatchLow** below, j=0 which defines the first comparator. If the developer wants to support multiple Context ID matches, then j=1,2,... defines additional comparators.

Table 20. Context ID Filtering

Register	Register Fields	Description
trTeComp0Control	**.trTeCompPInput** **.trTeCompSInput**	[1:0] Set = 1: context [3:2] Set = 1: context
	.trTeCompPFunction **.trTeCompSFunction**	[6:4] Set = 0: **equal to** trTeCompPMatch [10:8] Set = 0: **equal to** trTeCompSMatch
	.trTeCompMatchMode	[13:12] Set = 0: **primary result true**
	trTeCompPNotify	Set = 1 to generate an Event Trace packet when primary comparator match occurs
	All other fields set to 0	
trTeComp0MatchLow	**.trTeCompPMatchLow**	[31:0] **Match value for** (primary comparator)

(Secondary comparator is not needed for context ID filtering since it is only a single value match. It is unclear whether these 'S' fields need to be included in the hardware).

B.3. Exception Cause Register Filtering

Table 21. Exception Cause (Ecause) Filtering

Register	Register Fields	Description
trTeFilteriControl	**.trTeFilterEnable** **.trTeFilterMatchEcause**	Set = 1 Set = 1 (to compare Ecause value) All other fields set to 0
trTeFilteriMatchInst	**.trTeFilterValueInterrupt** (Note the name doesn't match the comparison)	[8:8] Set = 0 to match on exception (itype = 1) All other fields set to 0
trTeFilteriMatchEcauseLow	**.trTeFilterMatchChoiceEcauseLow**	[31:0] bit mask; set bit(s) to 1 to match encoded value of Ecause
trTeFilteriMatchEcauseHigh	**.trTeFilterMatchChoiceEcauseHigh**	[63:32] bits of Ecause (high) value

Table of (encoded) Exception cause values (from RISC-V Instruction Set Manual: Vol II).

Table 22. Exception cause values

Value	m-cause	s-cause	vs-mode	Value	m-cause	s-cause	vs-mode
0	Instruction address misaligned	Instruction address misaligned	Instruction address misaligned	13	Load page fault	Load page fault	Load page fault
1	Instruction access fault	Instruction access fault	Instruction access fault	14	Reserved	Reserved	Reserved
2	Illegal instruction	Illegal instruction	Illegal instruction	15	Store/AMO page fault	Store/AMO page fault	Store/AMO page fault
3	Breakpoint	Breakpoint	Breakpoint	16	Double trap	Reserved	Reserved

Value	m-cause	s-cause	vs-mode	Value	m-cause	s-cause	vs-mode
4	Load address misaligned	Load address misaligned	Load address misaligned	17	Reserved	Reserved	Reserved
5	Load access fault	Load access fault	Load access fault	18	Software check	Software check	Reserved
6	Store/AMO address misaligned	Store/AMO address misaligned	Store/AMO address misaligned	19	Hardware error	Hardware error	Reserved
7	Store/AMO access fault	Store/AMO access fault	Store/AMO access fault	20	Reserved	Reserved	Instruction guest-page fault
8	Environment call from U-mode	Environment call from U-mode	Environment call from U-mode or VU-mode	21	Reserved	Reserved	Load guest-page fault
9	Environment call from S-mode	Environment call from S-mode	Environment call from HS-mode	22	Reserved	Reserved	**Virtual instruction**
10	Reserved	Reserved	Environment call from VS-mode	23	Reserved	Reserved	Store/AMO guest-page fault
11	Environment call from M-mode	Reserved	Environment call from M-mode	24-31	Custom use	Custom use	Custom use
12	Instruction page fault	Instruction page fault	Instruction page fault	32-47	Reserved	Reserved	Reserved
				48-63	Custom use	Custom use	Custom use

B.4. Interrupt Cause Register Filtering

Note that several of the registers are "shared" with Ecause filtering, thus the name of the field may not match the comparison being made. For example, trTeFilterMatchChoiceEcauseLow is used to compare against the cause value for both exceptions and interrupts.

Table 23. Interrupt Cause Register Filtering

Register	Register Fields	Description
trTeFilteriControl	**.trTeFilterEnable** **.trTeFilterMatchInterrupt**	Set = 1 Set = 1 (to compare Interrupt cause value) All other fields set to 0
trTeFilteriMatchInst	**.trTeFilterValueInterrupt**	[8:8] Set = 1 to match on interrupt (itype = 2) All other fields set to 0
trTeFilteriMatchEcauseLow	**.trTeFilterMatchChoiceEcauseLow**	[31:0] bit mask; set bit(s) to 1 to match encoded values 0 to 31 of Interrupt cause
trTeFilteriMatchEcauseHigh	**.trTeFilterMatchChoiceEcauseHigh**	[63:32] bit mask; set bit(s) to 1 to match encoded values 32 to 63 of Interrupt cause

The (encoded) Interrupt cause values (from the RISC-V Instruction Set Manual: Vol II) are located in Table 16 - mcause, Table 37 - scause, and Table 50 when the hypervisor extension is implemented. Below is Table 50:

Exception Code	Description
0	Reserved
1	Supervisor software interrupt
2	Virtual supervisor software interrupt
3	Machine software interrupt
4	Reserved
5	Supervisor timer interrupt
6	Virtual supervisor timer interrupt
7	Machine timer interrupt
8	Reserved
9	Supervisor external interrupt
10	Virtual supervisor external interrupt
11	Machine external interrupt
12	Supervisor guest external interrupt
13	Counter-overflow interrupt
14-15	Reserved
≥16	Designated for platform use

Index

Bibliography

[1] RISC-V International, “RISC-V Hart to Trace Interface Specification,” RISC-V International, Version 0.83, Mar. 2026. [Online]. Available: github.com/riscv-non-isa/riscv-hart-trace-interface/releases.

[2] riscv-trace-control-interface-spec-1.0

[3] RISC-V International, “The RISC-V Instruction Set Manual, Volume II: Privileged Architecture,” RISC-V International, Official Release, Jan. 2026. [Online]. Available: docs.riscv.org/reference/isa/_attachments/riscv-privileged.pdf.

[4] RISC-V International, “The RISC-V Debug Specification,” RISC-V International, Version 1.0, Feb. 2025. [Online]. Available: github.com/riscv/riscv-debug-spec/releases/tag/1.0.

[5] RISC-V International, “RISC-V N-Trace Specification,” RISC-V International, Version 1.0, Nov. 2024. [Online]. Available: github.com/riscv-non-isa/riscv-nexus-trace/blob/main/pdfs/RISC-V-N-Trace.pdf.

[6] RISC-V International, “RISC-V E-Trace Specification,” RISC-V International, Version 2.0, Jun. 2025. [Online]. Available: docs.riscv.org/reference/e-trace/_attachments/riscv-trace-spec.pdf.